

Project Title: Agentic AI Assistant for Engineering Parametric CAD (Computer Aided Design)

Student Name: Ved Ravindra Prakash

Student ID: 3053362

Supervisor Name: Prof. Dr. Syed Fawad Hussain

Project Category: Artificial Intelligence

Project Aim:

The aim of this project is to build a local tool base which the locally hosted SLM models will use to provide an Agentic AI assistant which will help the user in creating, modifying and physically validating parametric 3D CAD models through natural – Language conversation, working as a collaborator inside an existing CAD application (FreeCAD). Every model the agent proposes is presented as an editable, step-by-step plan that the user approves before any geometry is created or edited and all reasoning and data remain on the user's own machine ensuring high data privacy. The outcome will be a working AI assistant plus an empirical study of how a fully local large-language-model (LLM) agent performs at assisted CAD design.

Significance: CAD is slow to learn and demands precise, ordered operations, so an assistant that translates intent into verifiable modelling steps could lower this barrier substantially. Crucially, engineering designs are confidential, yet today's AI CAD tools are cloud services; a system that assists usefully without sending any data off the device meets a real, unmet need.

Relevance: The core of the work is applied artificial intelligence: LLM-based planning and tool use, intent routing, retrieval-augmented memory with reflective consolidation, constrained structured generation for reliability, and the controlled, sandboxed generation of new tools by the agent itself. The CAD software is already available and the Agent will act as an independent wrapper around it to support CAD designing.

Related Available Works:

- **Generative CAD models.** DeepCAD (Wu et al., 2021) and Text2CAD (Khan et al., 2024) learn to emit CAD construction sequences from data, but require training and produce one-shot results with no user checkpoint.
- **LLM-driven CAD scripting.** Query2CAD (Badagabettu et al., 2024) shows LLMs can write executable FreeCAD macros and Seek-CAD (2025) explores local inference, yet both generate code without an interactive verification step.
- **Agentic CAD frameworks.** Recent agents (an ICED 2025 framework; CADDesigner, 2025) add tool calling and visual feedback, and report that LLMs struggle with spatial reasoning — motivating a human confirmation step over full autonomy.
- **Agent foundations.** ReAct (2023), Reflexion (2023), Voyager (2023), and Generative Agents (2023) supply the reasoning-and-acting loop, self-correction, skill accumulation, and memory consolidation patterns this project adapts.

Project Deliverables:

Sr. No.	Objective	Outcome
1	Integrate an LLM agent with a parametric CAD tool via the Model Context Protocol (MCP), exposing core modelling operations.	Agent creates a specified part (e.g. a holed block) end-to-end from a chat instruction.
2	Implement a plan–confirm–execute loop with per-step geometric validation and bounded error recovery, plus file ingestion and edit-by-description.	target part is produced within tolerance; interventions and auto-recovery rate are logged.
3	Provide physics validation (linear-static FEA and steady-state thermal) as agent-callable tools.	Agent sets up and solves a cantilever case like problems and reports max stress and a safety factor.
4	Build an isolated testing space “toolroom” for safe agent self-extension, and a reflective memory that consolidates and prunes interaction history.	Two agent-generated tools verified against analytical answers. Memory on/off will show measurable effect.

Methodology:

The CAD kernel and interface are reused, not rebuilt: FreeCAD (open-source, fully Python-scriptable) supplies parametric modelling, multi-format import/export, and a validated finite-element workbench. A small local model (served by Ollama) routes each request and a larger local model acts as planner; modelling, simulation, ingestion, and the toolroom are each exposed as isolated MCP tool-server processes. Reliability is engineered, not assumed: plans use constrained JSON generation so they are never malformed; the agent inspects the live model (feature tree, dimensions, screenshots) and the kernel validates every executed step, feeding errors back for a bounded number of repair attempts. Geometry from uploaded STEP/STL files is parsed deterministically to avoid hallucination, while drawing/PDF interpretation is assistive and user-verified. Memory works by logging episodes, periodically distilling them into compact “lesson cards” in a local vector store, retrieving the most relevant per query, and pruning consolidated logs to bound growth.

Project Plan:

Sr. No.	Task	Duration
1	Foundation, FreeCAD scripting and tools creation for generating a CAD model in FreeCAD software	6 Days
2	MCP server for CAD operations and agent Loop	4 Days
3	Plan – Confirm – Execute Loop functioning, different file ingestion and analysis pipelines and model tuning	8 Days
4	Simulation Tools addition (FEA + Thermal) and GUI setup for Viewing Physics Results	6 Days
5	Complete toolroom setup and reflective memory adaptation	4 Days
6	Routing and privacy verification	4 Days
7	Evaluation, testing, validation and refinement	8 Days

Risk & Contingency Plan:

Risks / Difficult Aspect	Contingency / Handling
Local LLM produces unreliable or malformed tool calls.	Constrained JSON output, small atomic tools, per-step kernel validation, and the confirmation step catch errors before they reach the model.
GPU memory cannot host the intended planner model.	Validate on day one; fall back to smaller quantised models (the design is model-agnostic via Ollama).
Drawing/PDF understanding by local vision models is weak (the aspect I am most cautious about).	Scope it as assistive and user-verified to rely on deterministic STEP/STL parsing for exact geometry.
Schedule slips.	Fixed de-scope order — drop PDF understanding, then thermal analysis, then the separate router — while protecting the plan–confirm loop and memory ablation; three-day buffer.
Agent-generated code is unsafe.	Static checks, a no-network sandbox with resource limits, and mandatory human approval before any tool is registered.

Hardware / Software Resources:

Hardware: A PC/laptop with a CUDA-capable GPU (≥ 6–12 GB VRAM) for local inference, which I have access to (larger models can be quantised to fit).

Software (all free / open-source): Python 3.11+, FreeCAD 1.0 with its FEM workbench, the MCP Python SDK, Ollama with open-weight models (e.g. Qwen2.5), SQLite, and ChromaDB. No paid service is needed. An optional cloud LLM for better reasoning is there but it is off by default.

Data:

The system is training-free, so no large dataset is required. Test models come from public, openly licensed CAD datasets (e.g. DeepCAD, Fusion 360 Gallery). Memory data is generated locally at runtime. There are therefore no data-access, licensing, or privacy obstacles and no third-party data dependency.